

CHAPTER -FOUR

Libraries and Framework

MVC

What is MVC?

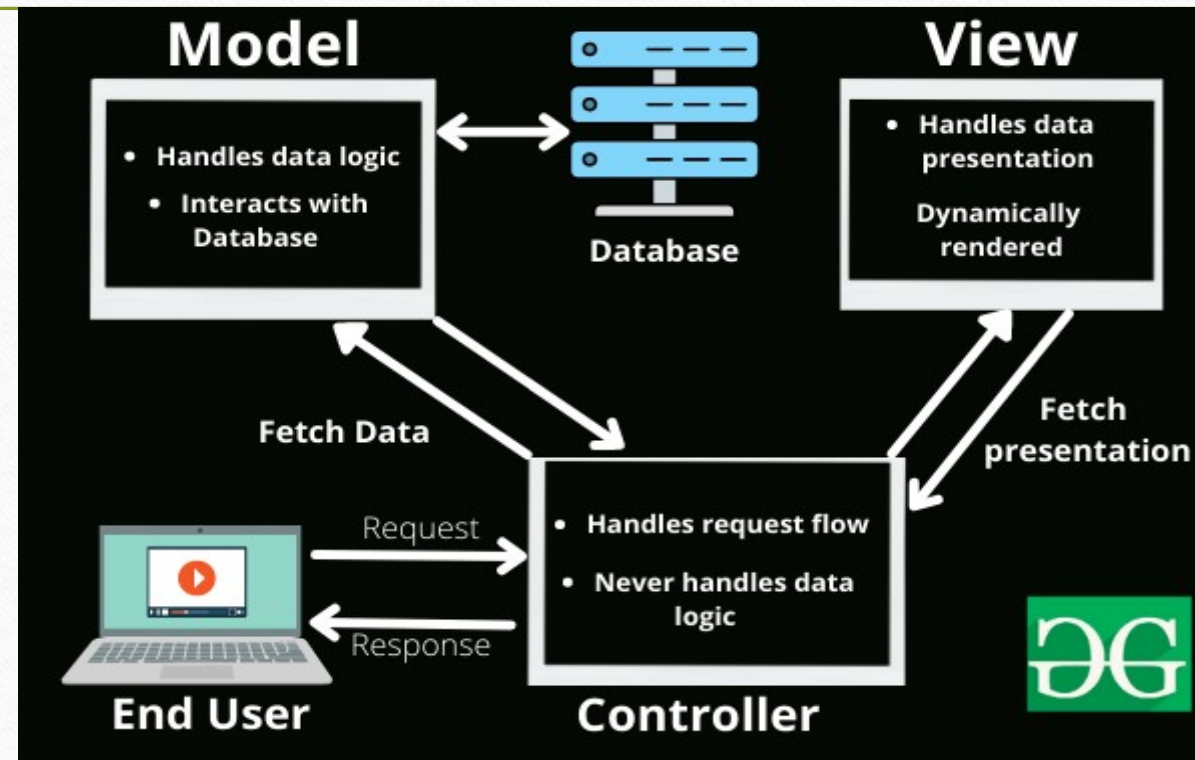
- The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller.
- Each architectural component is built to handle specific development aspects of an application.
- It isolates the business logic and presentation layer from each other.
- It was traditionally used for desktop graphical user interfaces (GUIs).
- Nowadays, MVC is one of the most frequently used industry-standard web development frameworks to create scalable and extensible projects.
- It is also used for designing mobile apps.

Features of MVC

- It provides a clear separation of business logic, UI logic, and input logic.
- It offers full control over your HTML and URLs which makes it easy to design web application architecture.
- It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
- It supports Test Driven Development (TDD).

Components of MVC

- Controller
- Model
- View



MVC Architecture Design

Controller

- The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary.
- The controller doesn't have to worry about handling data logic, it just tells the model what to do.
- It processes all the business logic and incoming requests, manipulates data using the Model component, and interact with the View to render the final output.

Responsibilities:

- Receiving user input and interpreting it.
- Updating the Model based on user actions.
- Selecting and displaying the appropriate View.

Example: In a bookstore application, the Controller would handle actions such as searching for a book, adding a book to the cart, or checking out.

View

- The View component is used for all the UI logic of the application.
- It generates a user interface for the user.
- Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller.
- It only interacts with the controller.

Responsibilities:

- Rendering data to the user in a specific format.
- Displaying the user interface elements.
- Updating the display when the Model changes.

Example: In a bookstore application, the View would display the list of books, book details, and provide input fields for searching or filtering books.

Model

- The Model component corresponds to all the data-related logic that the user works with.
- This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.
- It can add or retrieve data from the database.
- It responds to the controller's request because the controller can't interact with the database by itself.
- The model interacts with the database and gives the required data back to the controller.

Responsibilities:

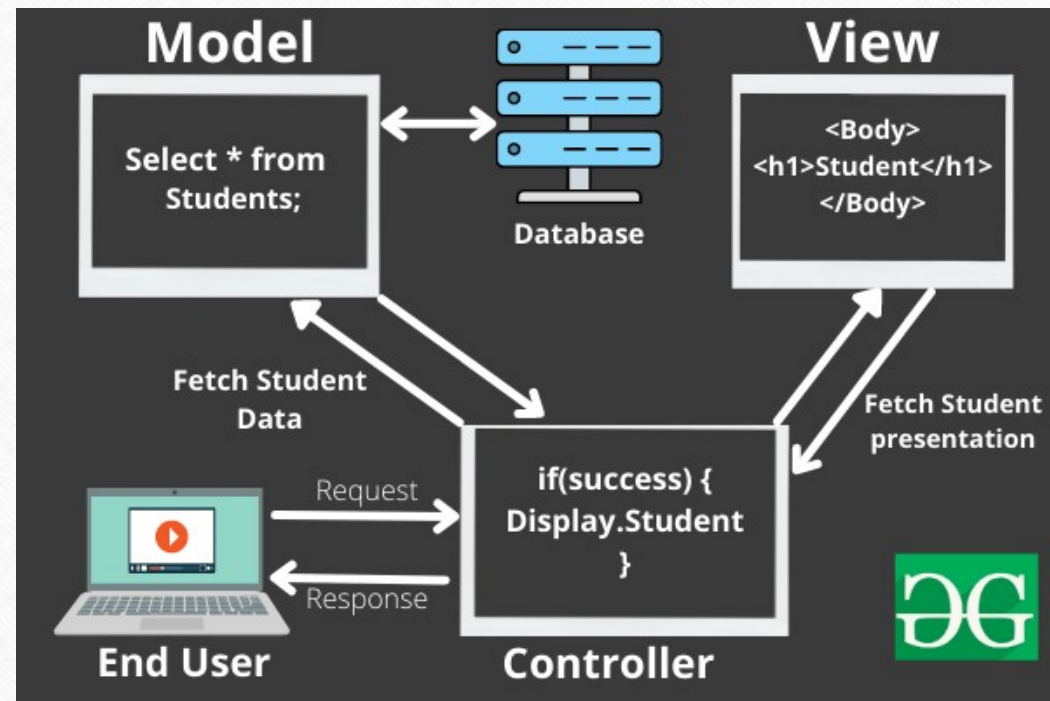
- Managing data: CRUD (Create, Read, Update, Delete) operations.
- Enforcing business rules.
- Notifying the View and Controller of state changes.

Example: In a bookstore application, the Model would handle data related to books, such as the book title, author, price, and stock level.

Working of the MVC framework with Example

- Let's imagine an end-user sends a request to a server to get a list of students studying in a class.
- The server would then send that request to that particular controller that handles students.
- That controller would then request the model that handles students to return a list of all students studying in a class.

Working of the MVC framework with Example



The flow of Data in MVC Components

Cont...

- The model would query the database for the list of all students and then return that list back to the controller.
- If the response back from the model was successful, then the controller would ask the view associated with students to return a presentation of the list of students.
- This view would take the list of students from the controller and render the list into HTML that can be used by the browser.
- The controller would then take that presentation and returns it back to the user.
- Thus ending the request. If earlier the model returned an error, the controller would handle that error by asking the view that handles errors to render a presentation for that particular error.
- That error presentation would then be returned to the user instead of the student list presentation.

The MVC architectural pattern

1. **Divide and conquer.** The three components can be somewhat independently designed.
2. **Increase cohesion.** The components have stronger layer cohesion than if the view and controller were together in a single UI layer.
3. **Reduce coupling.** The communication channels between the three components are minimal and easy to find.
4. **Increase reuse.** The view and controller normally make extensive use of reusable components for various kinds of UI controls. The UI, however, will become application-specific, therefore it will not be easily reusable.
5. **Design for flexibility.** It is usually quite easy to change the UI by changing the view, the controller, or both.

Package Manager

- A package is a broader concept that encompasses libraries along with additional metadata about the software.
- Packages are typically distributed via package managers, which help manage dependencies and versions effectively.
- In PHP, popular package managers like Composer enable developers to include and manage libraries easily.

Package Manager

A Package Manager is a tool that automates the process of managing software packages and dependencies.

- It allows you to easily install, update, and manage libraries and frameworks for your projects.
- In PHP, the most commonly used package manager is Composer.

Composer:

- Composer is a dependency manager for PHP, which helps you manage libraries and their dependencies. It downloads and installs PHP packages from a central repository (Packagist).
- Composer uses a file called `composer.json` to define dependencies and their versions.

Basic Composer Commands:

- **Install dependencies:** `composer install`
- **Install a specific package:** `composer require package-name`
- **Update packages:** `composer update`

Example: Install a Package Using Composer:

`composer require monolog/monolog`

This will download and install the Monolog logging library.

Libraries

- A library is a collection of pre-written code that performs a specific function.
- It might be used to handle tasks like sending emails, logging, or interacting with a database.

Example Libraries:

PHPMailer: For sending emails.

Guzzle: HTTP client for making requests.

Carbon: A library for working with dates and times.

PHP Libraries

1. Composer

- Composer is a dependency manager for PHP.
- It is used to manage the packages and libraries that your project depends on.
- Composer allows you to easily add and update dependencies to your project with just a few commands.
- It also provides a central repository where you can search for and download packages.

PHP Libraries

2. Guzzle

- Guzzle is a PHP HTTP client that makes it easy to send HTTP requests and handle responses.
- It supports features like authentication, caching, and async requests.
- Guzzle is widely used in PHP projects to communicate with APIs and other web services.

3. Monolog

- Monolog is a logging library for PHP.
- It provides a flexible and extensible logging system that can write to various outputs like files, syslog, email, and more.
- Monolog is easy to use and highly customizable, making it a popular choice for logging in PHP applications.

PHP Libraries

4. PHPUnit

- PHPUnit is a unit testing framework for PHP.
- It provides a set of tools for writing and running automated tests to ensure that your code is functioning correctly.
- PHPUnit is widely used in PHP development to improve code quality and catch bugs early.

5. Swift Mailer

- Swift Mailer is a feature-rich email library for PHP.
- It provides a simple and flexible API for sending emails, supporting features like attachments, HTML emails, and SMTP authentication.
- Swift Mailer is a popular choice for sending transactional emails and newsletters.

PHP Libraries

6. PHPMailer

- PHPMailer is another popular email library for PHP.
- It provides similar features to Swift Mailer, but also supports features like mailing list management, email templates, and bounce handling.
- PHPMailer is a great choice for complex email projects.

Features of PHP Libraries

Modularity:

- Most libraries are designed to be modular, meaning that you can include only the components you need for your specific application.
- This modularity not only keeps your codebase clean but also reduces the overall size of your application.

Version Control:

- Libraries often come with version control, allowing developers to maintain compatibility with their applications.
- Package managers like Composer make it easy to specify which version of a library your project requires, ensuring that updates do not break existing functionality.

Autoloading:

- PHP libraries typically support autoloading, which automatically loads the required classes when they are needed.
- This feature helps to reduce the amount of code you need to write and keeps your application performance optimized.

The Role of Libraries in PHP Development

- **Code Reusability:** is the ability to reuse code.
 - This not only speeds up the development process but also promotes cleaner code practices.
 - By leveraging existing libraries, developers can focus on the unique aspects of their applications rather than getting bogged down by repetitive tasks.
- **Community Support:**
 - Many PHP libraries are open-source and come with a vibrant community of developers.
 - This community support can be invaluable for troubleshooting issues, sharing enhancements, or even contributing to the library's future development.

The Role of Libraries in PHP Development

- **Best Practices:**

- Libraries are often developed by experienced professionals who adhere to industry standards and best practices.
- By using well-established libraries, developers can ensure that their code is secure and performs optimally.

- **Integration with Frameworks:**

- PHP frameworks like Laravel, Symfony, and CodeIgniter often come with their own set of libraries.
- These libraries are optimized for use within the framework, making it easier to build applications that are efficient and maintainable.

PHP Frameworks

- A **framework** is a larger, more structured set of tools and libraries that provides a complete structure to build an entire application.

1. Laravel

- Laravel is a popular PHP web application framework.
- It follows the Model-View-Controller (MVC) architecture and provides many built-in features like routing, middleware, authentication, and more.
- Laravel is highly extensible and customizable, making it a great choice for building complex web applications.

PHP Frameworks

2. Symfony

- Symfony is another popular PHP web application framework.
- It is highly modular and provides many reusable components like HTTP handling, form handling, and templating.
- Symfony also follows the MVC architecture and provides many built-in features like routing, security, and caching.

3. CodeIgniter

- CodeIgniter is a lightweight PHP web application framework.
- It is easy to learn and provides many features like database abstraction, form validation, and security.
- CodeIgniter is a great choice for small to medium-sized projects that do not require the complexity of larger frameworks like Laravel or Symfony.

PHP Frameworks

4. CakePHP

- CakePHP is a popular PHP web application framework that follows the MVC architecture.
- It provides many built-in features like database abstraction, authentication, and security.
- CakePHP is easy to learn and has a low learning curve, making it a great choice for beginners.

5. Zend Framework

- Zend Framework is a mature and robust PHP web application framework.
- It provides many reusable components like authentication, caching, and logging.
- Zend Framework follows the MVC architecture and is highly customizable, making it a great choice for large and complex web applications.

Purpose of a Framework:

- To standardize and streamline web development.
- Provide built-in tools for routing, database handling, validation, templating, etc.
- Help developers follow best practices and design patterns like MVC (Model-View-Controller).

Laravel Project File Structure

laravel-project/

├── app/

├── bootstrap/

├── config/

├── database/

├── public/

├── resources/

├── routes/

├── storage/

├── tests/

├── .env

├── .gitignore

├── artisan

├── composer.json

├── package.json

├── phpunit.xml

└── README.md

Key Directories

1. app/

- Contains the core application code.
- Subfolders: Http/, Models/, Console/, Providers/
- You write controllers, middleware, models, and service providers here.

2. bootstrap/

- Contains the app.php file which bootstraps the framework.
- Also contains the cache/ directory for framework-generated files.

3. config/

- Holds configuration files for all parts of Laravel (app.php, database.php, mail.php, etc.)

4. database/

- For database-related files: migrations, seeders, and factories.

Key Directories

5. public/

- The document root (publicly accessible directory).
- Contains index.php, which is the entry point for all requests.

6. resources/

- Views (views/), raw assets like CSS/JS (css/, js/), and localization files.

7. routes/

- Defines routes using web.php, api.php, console.php, and channels.php.

8. storage/

- For logs, cache, compiled Blade templates, file uploads, etc.

9. tests/

- For automated tests (PHPUnit or Pest).

Composer Integration (vendor/)

- Composer installs all Laravel dependencies and other third-party packages into the vendor/ directory.

Example structure:

vendor/

├── autoload.php

├── composer/

├── laravel/

│ ├── framework/ ← Core Laravel framework

├── symfony/

│ ├── components used by Laravel (e.g. HttpFoundation, Console)

├── psr/

│ ├── interfaces and standards (e.g. log, container, http-message)

├── monolog/ ← Logging

├── nesbot/

│ ├── carbon/ ← Date handling

composer.json (Project Manifest)

Defines your app's dependencies, autoloading, and scripts:

```
{
  "require": {
    "php": "^8.1",
    "laravel/framework": "^10.0",
    "guzzlehttp/guzzle": "^7.0"},
  "autoload": {
    "psr-4": {
      "App\\": "app/"
    }
  }
}
```

After modifying this file, running `composer install` or `composer update` updates the `vendor/` directory and `composer.lock`

Laravel + Composer Flow

- Composer installs dependencies → into vendor/
- Laravel autoloads classes using Composer's autoload.php
- Your code uses those classes transparently, e.g., `Illuminate\Http\Request`

Framework as a Package

- Laravel itself is modular and composed of many packages:
- laravel/framework (meta-package, pulls in all needed components)
- Subcomponents are based on Symfony + custom Laravel implementations